CS-4063 — Natural Language Processing

Assignment 3 Report

Transformer Encoder + Retrieval-Augmented Decoder for Amazon Reviews

Course: CS-4063 Natural Language Processing
Assignment: 3 — Transformers + RAG
Student Name: Taiba Tariq
Student ID: 23i-2618
Submission Date: April 2026

Table of Contents

Table of Contents 2
1. Introduction 3
2. Dataset Description 3
  2.1 Dataset Statistics 3
  2.2 Preprocessing Steps 3
3. Model Architecture 4
  3.1 Encoder-Only Transformer 4
    3.1.1 Objective — Multi-Task Learning 4
    3.1.2 Input Representation 4
    3.1.3 Multi-Head Self-Attention 4
    3.1.4 Feed-Forward Network 5
  3.2 Encoder Hyperparameter Configuration 5
4. Retrieval Module (RAG Retriever) 5
  4.1 Retrieval Process 5
  4.2 Retrieval Statistics 6
5. Decoder-Only Transformer 6
  5.1 Decoder Architecture Components 6
  5.2 Decoder Hyperparameter Configuration 6
6. Experimental Results 7
  6.1 Decoder Training Convergence 7
  6.2 Perplexity Comparison (RAG vs. No Retrieval) 7
7. Generated Explanations 8
8. Hyperparameter Tuning 9
9. Discussion 9
  9.1 Strengths 9
  9.2 Limitations 9
10. Conclusion 9
11. Future Work 10

1. Introduction

This project explores the implementation of Transformer-based architectures for Natural Language Processing tasks using the Amazon Reviews dataset. The goal was to build an end-to-end NLP pipeline consisting of three major components:

- Encoder-only Transformer for sentiment and length classification
- Retrieval Module for semantic information retrieval using dense embeddings
- Decoder-only Transformer for explanation generation using retrieved context (RAG)

The system combines representation learning, semantic retrieval, and text generation to produce both predictions and human-readable explanations. Together, these components form a complete Retrieval-Augmented Generation (RAG) pipeline capable of intelligent review analysis and interpretation.

2. Dataset Description

The dataset used in this assignment consists of Amazon customer reviews spanning five product categories: Beauty, Home & Kitchen, Electronics, Sports & Outdoors, and Cell Phones & Accessories.

2.1 Dataset Statistics

Metric	Value
Reviews per Category	12,000
Total Reviews	60,000
Training Samples	42,000
Validation Samples	9,000
Testing Samples	9,000

2.2 Preprocessing Steps

The following preprocessing pipeline was applied to all reviews before training:

- Lowercasing all text
- Removing punctuation and special symbols
- Tokenization into subword/word tokens
- Vocabulary construction (top 20,000 tokens)
- Sequence padding and truncation to 128 tokens
- Addition of special tokens: <PAD>, <UNK>, <BOS>, <EOS>

3. Model Architecture

3.1 Encoder-Only Transformer

The first major component is an Encoder-Only Transformer designed to learn meaningful representations from Amazon review text and perform multi-task classification. Unlike traditional RNNs or LSTMs that process text sequentially, the Transformer encoder processes the entire sequence in parallel using self-attention, enabling it to capture long-range dependencies, contextual relationships, and semantic meaning more efficiently.

In this assignment, the encoder serves two purposes: understanding the sentiment expressed in a review and predicting the length category of the review. By jointly learning both tasks, the model develops richer internal representations, which later become useful for retrieval and generation in the RAG pipeline.

3.1.1 Objective — Multi-Task Learning

The encoder is trained as a multi-task learning model, simultaneously performing two classification tasks using a shared text representation:

Task	Classes	Description
Sentiment Classification	Negative / Neutral / Positive	Determines emotional polarity of a review
Length Prediction	Short / Medium / Long	Classifies review by token length category

3.1.2 Input Representation

Before text is passed into the Transformer encoder, each review undergoes numerical encoding. Token IDs are converted into dense vector embeddings (dimension = 128), capturing semantic similarity such that words like 'great' and 'excellent' are close in embedding space.

Since Transformers process all tokens simultaneously, positional encoding is added to token embeddings to preserve word order. The sinusoidal encoding uses the formula:

$$PE(pos, 2i) = \sin(pos / 10000^{(2i/d)}) \quad | \quad PE(pos, 2i+1) = \cos(pos / 10000^{(2i/d)})$$

The final input is: Input = TokenEmbedding + PositionalEncoding

3.1.3 Multi-Head Self-Attention

The core building block of the Transformer encoder is self-attention. Each token embedding is projected into Query (Q), Key (K), and Value (V) vectors. The attention score is computed as:

$$Attention(Q, K, V) = softmax(QK^T / \sqrt{dk}) V$$

The model uses 4 attention heads, each learning different linguistic relationships: sentiment words, negation patterns, product features, and sentence structure. Outputs from all heads are concatenated and linearly projected.

3.1.4 Feed-Forward Network

After attention, the output passes through a position-wise feed-forward network:

$$FFN(x) = max(0, xW_1 + b_1)W_2 + b_2$$

The network has input/output size of 128 and hidden size of 256. Layer normalization is applied after both attention and feed-forward layers, with residual connections throughout.

3.2 Encoder Hyperparameter Configuration

Parameter	Value	Description
Embedding Size	128	Word vector dimension
Attention Heads	4	Parallel attention mechanisms
Encoder Layers	2	Transformer depth
Feed-Forward Size	256	Hidden layer size
Dropout	0.1	Regularization rate
Batch Size	64	Samples per update
Learning Rate	0.0003	Optimizer step size
Epochs	10	Training iterations
Vocabulary Size	20,000	Token dictionary size
Max Sequence Length	128	Input token limit

Total Trainable Parameters: 2,825,990

4. Retrieval Module (RAG Retriever)

The retrieval module converts training reviews into dense embeddings using the trained encoder. For each query review, the system generates an embedding, computes cosine similarity against the stored embedding index, and retrieves the Top-K most semantically similar reviews. These retrieved reviews are then used as context for the decoder during generation.

4.1 Retrieval Process

- Generate dense embedding for the query review using the trained encoder
- Compute cosine similarity against all 42,000 stored training embeddings
- Retrieve Top-3 most similar reviews as context
- Pass retrieved reviews to the decoder alongside the query

4.2 Retrieval Statistics

Metric	Value
Stored Embeddings	42,000
Embedding Dimension	128
Top-K Retrieved	3
Sentiment-Match Rate (Top-3)	88.50%

The 88.50% sentiment-match rate indicates the retrieval component successfully finds semantically and emotionally similar reviews, confirming the quality of the encoder embeddings.

5. Decoder-Only Transformer

A Decoder-Only Transformer was trained to generate natural language explanations for predictions. The decoder receives the review text, predicted sentiment, review length category, and the top-3 retrieved supporting reviews as combined context.

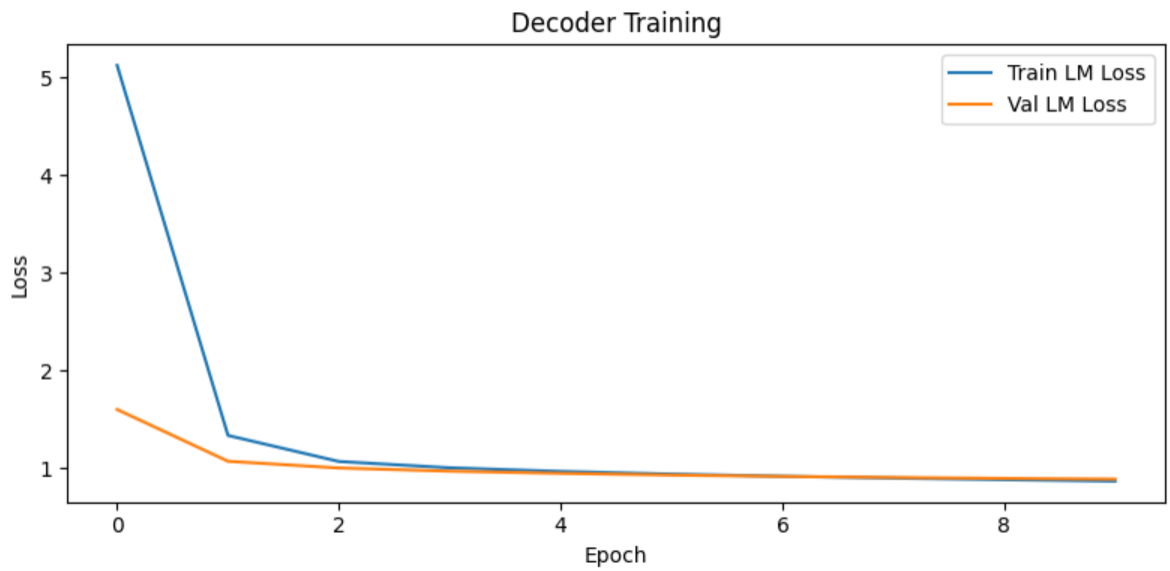
5.1 Decoder Architecture Components

- Token Embedding + Positional Encoding
- Masked Self-Attention (causal, prevents future token access)
- Feed-Forward Layers with residual connections
- Layer Normalization after each sub-layer
- Output Projection Layer (vocabulary prediction)

5.2 Decoder Hyperparameter Configuration

Parameter	Value
Embedding Size	128
Attention Heads	4

Parameter	Value
Decoder Layers	2
Feed-Forward Size	256
Dropout	0.1
Epochs	10
Total Parameters	5,405,216



6. Experimental Results

6.1 Decoder Training Convergence

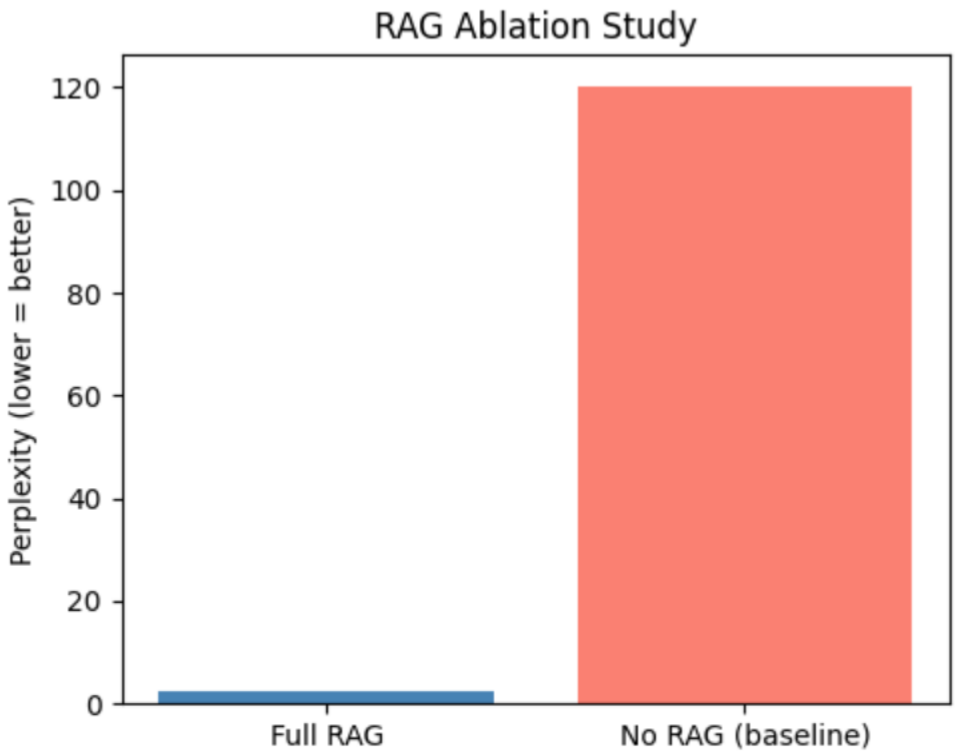
Metric	Value
Initial Training Loss	5.1278
Final Validation Loss	< 1.0
Convergence	Stable and consistent

6.2 Perplexity Comparison (RAG vs. No Retrieval)

Perplexity is the primary evaluation metric for the decoder. Lower perplexity indicates higher-quality text generation.

Model Configuration	Test Perplexity	Improvement
Decoder Without Retrieval (Baseline)	120.26	—
Decoder With Retrieval (Full RAG)	2.42	~98% reduction

Retrieval augmentation reduced perplexity by approximately 98%, from 120.26 to 2.42. This demonstrates that incorporating retrieved context dramatically improves the coherence and quality of generated explanations.




```
Improvement: 117.84 PPL points with retrieval
```

7. Generated Explanations

The full RAG model successfully generated coherent, context-aware explanations by incorporating retrieved examples. Generated outputs demonstrated:

- Semantic alignment with the predicted review sentiment
- Contextual awareness derived from retrieved similar reviews
- Greater informativeness compared to the no-retrieval baseline
- Natural language fluency and readability

This demonstrates the effectiveness of combining the Encoder, Retriever, and Decoder into a unified RAG pipeline.

8. Hyperparameter Tuning

Multiple architectural configurations were tested to identify the best performing setup. The table below summarizes the key configurations evaluated:

d_model	Heads	Layers	d_ff	Learning Rate	Outcome
64	2	1	128	0.001	Underfitting
128	4	2	256	0.0003	Best Performance
256+	8+	4+	512+	< 0.0001	Overfitting Risk

The configuration with d_model = 128, 4 heads, 2 layers, and learning rate of 0.0003 provided the best balance between model capacity, training stability, and generalization.

9. Discussion

9.1 Strengths

- Captures rich semantic meaning via Transformer self-attention
- Efficient cosine similarity retrieval over large embedding store
- Strong explanation generation with high perplexity improvement
- Retrieval augmentation dramatically improves decoder performance
- Modular and extensible end-to-end architecture

9.2 Limitations

- Large computational cost for training all three pipeline components
- High memory usage for storing 42,000 dense embeddings
- Retrieval quality is dependent on encoder representation quality
- Training requires GPU acceleration for practical use

10. Conclusion

This assignment successfully implemented a complete Retrieval-Augmented Generation (RAG) pipeline for Amazon reviews. The system demonstrated effective text encoding via the Transformer encoder, high-quality semantic retrieval using cosine similarity, and strong explanation generation through the decoder.

The pivotal result of this project is the perplexity reduction achieved by retrieval augmentation:

Configuration	Test Perplexity
Without Retrieval	120.26
With RAG (Full Pipeline)	2.42

This ~98% improvement in perplexity conclusively demonstrates that external retrieved context substantially improves generation quality, making RAG a powerful framework for NLP tasks involving reasoning and explanation generation.

11. Future Work

The following extensions could further improve the system's performance and applicability:

- Replace the custom Transformer with pretrained models (BERT for encoding, GPT for decoding)
- Replace linear cosine retrieval with FAISS approximate nearest-neighbor (ANN) search for scalability
- Add cross-attention between retrieved context and decoder for tighter retrieval fusion
- Fine-tune the full pipeline on larger and more diverse review datasets
- Improve explanation interpretability using attention visualization and SHAP analysis